

Statut d'implémentation — Sentinel-Guard v4

Dernière mise à jour : voir date du commit. 220 tests passent (python -m pytest).

Le delta v3 → v4 (mode document §7ter, piège B5, INV-015/016/017, mesure par mode, types v4) est implémenté — voir la section « Delta v4 » ci-dessous.

Estimation globale : ~87% de la spec v3 implémentée (+ delta v4 complet). Le cœur de la garantie (§2, §4, §6, §7, §14, §15) est complet et vérifié en conditions réelles. L'infrastructure de mesure (§12), y compris la calibration inter-annotateur, est codée et testée ; il ne manque que de vraies annotations humaines pour l'exercer en conditions réelles. Le seul point structurellement hors de portée du code seul reste D1-D3/C3/E3, que la spec elle-même classe limite mathématique du contrôle déterministe (§10).

Par section

Section	%	Module(s)	État
§1bis Glossaire	100%	types.py	Les 5 ClaimStatus respectent strictement les 3 niveaux fidélité/pertinence/interprétation
§2 Plancher de risque	~98%	triage.py, orchestration.py	max(LLM, plancher) automatique : couverture (E4), slot inféré (A3), troncature (B2), pertinence non garantie (C1), statut INTERPRÉTATION/CITÉ_NON_OPPOSABLE (B3/D1/F1), sélection ambiguë (A3 variante), query partagée entre N intentions (E1 dégradé) — aucun n'est contournable par l'appelant
§3 Abstraction moteur	~95%	llm.py, gemini.py, mistral.py, litellm_provider.py	Gemini + Mistral réels (wrappers HTTP maison), plus LiteLLMModelProvider (§17.1) — trois implémentations concrètes de ModelProvider, toutes vérifiées en direct
§4 Flux complet	100%	orchestration.py, core.py, coverage.py,	Toutes les étapes 0-10 implémentées et testées, multi-saut réel inclus

Section	%	Module(s)	État
		multi_hop.py, human_validation.py	
§4bis Structuré/ texte libre	~90%	moulineuse.py, datagouv.py, slot_provenance.py	Routes structurées + repli texte libre marqué "pertinence non garantie"
§4ter Multi- saut	100%	multi_hop.py, retrieval.py	Prouvé en direct sur 2 hops réels (Code civil → Code de la construction)
§5 Rôle LLM	~65%	llm.py	Mécanique opérationnelle avec vrais LLM ; texte exact des 4 règles du prompt B pas repris littéralement
§6 Forçage récupération	 100%	orchestration.py, llm.py	Satisfait par choix architectural : l'orchestrateur appelle toujours MCP directement (§4 étape 4), jamais le LLM — c'est la voie "backend local sans forçage" explicitement autorisée par §6 comme équivalente au forçage natif, appliquée à tous les backends. PromptBasedDecomposer/ PromptBasedIntentGenerator n'ont donc aucun outil à forcer, code nettoyé en conséquence
§6bis Moulineuse	~85%	moulineuse.py	3 routes prouvées en direct (code_article, pastille refus, texte_libre) ; pastille jamais testée en direct sur un chemin heureux
§6ter Opposabilité/ data.gouv	~92%	datagouv.py, moulineuse.py, file_retrieval.py	Sources réelles branchées et prouvées en direct : normatif (Moulineuse), donnée tabulaire (data.gouv API), donnée non-tabulaire (fichiers CSV/ZIP téléchargés — couvre l'INSEE dont la plupart des ressources ne sont PAS dans l'API tabulaire)
§7 Vérificateur déterministe	~95%	verifier.py, normalization.py	Contiguïté (insensible casse/ ponctuation de bord), anti-épissage, opposabilité avec re-contrôle du cycle de vie (etat), anti-troncature, ancres dures négation/chiffres sur les INTERPRÉTATION
§7bis Refus	90%	exceptions.py, verifier.py	Levée systématique, jamais de devinette
§7ter Mode document (v4)	~90%	document.py, types.py, measurement.py	3 modes (analyse/synthèse/ production), segmentation par le code, couverture documentaire (INV-017/B5), planchers par mode (INV-016), aucun

Section	%	Module(s)	État
			statut agrégé (INV-015). Reste : lookup d'existence article/alinéa visé en mode production (nécessite une récupération, couvert en attendant par le plancher élevé inconditionnel)
§8 Registre	~90%	<code>audit.py</code>	<code>passage_hashes</code> rejouables, intégré dans la façade unifiée
§9 API vs local	N/A	—	Narratif, satisfait par construction (agnostique au provider)
§10 Matrice des pièges	~70%	multiples	A1,A2,A3,B1,B2,B4,C1,C2,E1,E2,E4,F1 traités. D1-D3,C3,E3 hors de portée du code seul (jugement humain par construction, cf. §10 de la spec)
§12 Mesure	~95%	<code>measurement.py</code> , <code>trap_dataset.py</code> , <code>calibration.py</code>	Banc réel sur vérificateur+triage : 100% blocage correct, 0% sur-refus, 0% faux négatifs. (v4) sur-refus mesuré par mode document. Kappa de Cohen inter-annotateur implémenté et testé (accord parfait, partiel, désaccord systématique) — reste à exercer avec de vraies annotations humaines sur le gold standard, ce qui est une activité humaine, pas un manque de code
§13 Déploiement souverain	~40%	<code>sovereign_log.py</code>	§13.4 (cloisonnement logs) codé et câblé dans <code>SentinelGuard</code> . §13.1-13.3,13.5 narratif/DSI, non codable
§14 Invariants	~85%	multiples	Majorité vérifiable par construction/ tests, INV-015/016/017 (v4) testés . INV-009 narratif, INV-014 pas de scan d'imports dédié
§15 Interfaces canoniques	~95%	<code>types.py</code>	Correspondance quasi exacte avec les pseudo-types de la spec, types v4 inclus (<code>DocumentMode</code> , <code>DocumentDraft</code> , <code>CoverageMap</code>)
§16 Priorité des règles	N/A	—	Respecté par construction du code
§17 Dépendances externes	~75%	<code>mcp_client.py</code> , <code>litellm_provider.py</code>	<code>mcp</code> épinglé. LiteLLM implémenté (<code>litellm + truststore</code> en optional-dependency <code>litellm</code>), vérifié en direct sur Mistral. <code>deepeval/ragas/doccano</code> absents par choix (cohérent INV-014, mesure hors-runtime)

Pièges de la matrice §10 — détail

Piège	Statut	Mécanisme
A1 — référence inventée	✓ verrouillé	lookup, verifier.py
A2 — prémisses fausses	✓ verrouillé	lookup échoue → refus
A3 — référence inférée	✓ borné	slot_provenance.py, élévation auto du risque
B1 — citation exacte	✓ verrouillé	contiguïté, verifier.py
B2 — citation tronquée	✓ borné	_detect_adjacent_truncation, élévation auto
B3 — paraphrase distordue	✓ borné	ellipsis → INTERPRÉTATION ; ancres dures négation/chiffres ; toute INTERPRÉTATION → risque élevé → humain
B4 — épissage de fragments	✓ verrouillé	règle de contiguïté stricte
B5 — synthèse à trou (v4)	✓ borné	couverture documentaire (document.py, INV-017) : toute unité segmentée par le code est couverte ou déclarée omise ; résidu (omission interne à une unité couverte) → humain
C1 — source hors-sujet	✓ borné	route texte_libre, pertinence_non_garantie
C2 — source périmée	✓ borné	ETAT/opposable à la récupération + re-contrôle de metadata["etat"] dans le vérificateur (défense en profondeur)
C3 — mauvaise juridiction	✗ non traité	nécessite corpus multi-juridictionnel réel
D1 — interprétation déguisée	✗ hors de portée	jugement humain par construction (spec §10)
D2 — synthèse fallacieuse	✗ hors de portée	idem, limite mathématique explicite
D3 — fausse attribution	✗ hors de portée	idem
E1 — N questions → 1 requête	✓ borné	décomposition ; mais §4 étape 3 (1 requête PAR intention) non implémentée → query partagée entre N>1 intentions force le risque élevé
E2 — multi-saut non suivi	✓ verrouillé	bornes §4ter, retrieval.py

Piège	Statut	Mécanisme
E3 — référent manquant	✗ non traité	nécessite un tour de dialogue, pas une fonction pure
E4 — intention oubliée	✓ borné	coverage.py, élévation auto du risque
F1 — verbatim non opposable	✓ verrouillé	dérivation opposable, CITÉ_NON_OPPOSABLE

Modules (src/sentinel_guard/)

Module	Rôle
types.py	Pseudo-types canoniques (§15)
normalization.py	Normalisation texte/numérique (§7)
verifier.py	Vérificateur déterministe (§7)
triage.py	Plancher de risque (§2)
coverage.py	Echo-back + contrôle de couverture (§4 étape 1bis)
slot_provenance.py	Discriminateur slot copié/inféré (§4bis, A3)
retrieval.py	Bornes multi-saut génériques (§4ter)
multi_hop.py	Sélection de renvois réels Moulineuse (§4ter)
mcp_client.py	Façade synchrone sur le SDK MCP
moulineuse.py	RetrievalProvider Moulineuse réel (§6bis) : 4 routes — code_article, pastille, texte_libre, parlement_question
datagouv.py	RetrievalProvider data.gouv API tabulaire (§6ter)
file_retrieval.py	RetrievalProvider fichiers CSV/ZIP non-tabulaires (§6ter) : téléchargement + parse défensivement (détection ZIP/encodage/séparateur), adressage de cellule par filtres multi-colonnes
multi_source.py	Aiguillage entre les deux sources
orchestration.py	Boucle principale (§4)
llm.py	Abstraction LLM, prompts, parsing JSON
gemini.py / mistral.py	Providers LLM réels (wrappers HTTP maison)
litellm_provider.py	Provider LLM via LiteLLM (§17.1), auto-détection du magasin de certificats OS
document.py	Mode document v4 (§7ter) : segmentation, couverture documentaire, planchers par mode
human_validation.py	Registre de décisions humaines (§4 étape 9)

Module	Rôle
audit.py	Registre de conformité, hashes rejouables (§8)
sovereign_log.py	Cloisonnement conformité/accès (§13.4)
core.py	Façade unifiée SentinelGuard
measurement.py / trap_dataset.py	Banc de mesure (§12)
calibration.py	Kappa de Cohen inter-annotateur, validation du gold standard (§12)
exceptions.py	Hiérarchie d'exceptions

Ce qui reste

Bugs d'honnêteté d'affichage — CORRIGÉS

Révélés par le test « arrêt Assemblée plénière du 14 avril 2006 » (voir ANALYSE_TEST_JURISPRUDENCE.md pour le détail complet). La garantie de fond était déjà intacte (aucune hallucination, non-pertinence signalée, résultat non publiable) ; c'était la **présentation** du résultat qui brouillait le message :

1. **_compliance_status renvoyait VALIDATED quand 0 claim de réponse était produit** (audit.py) — `verify_claims([])` donne `verbatim_check=PASS` car `all([])==True`. Corrigé : nouvel état `NO_ANSWER`, retourné en priorité quand `result.verifcation.claims` est vide. Test de régression : `test_compliance_status_is_no_answer_when_zero_claims`.
2. **Le claim de contrôle d'UI s'affichait même sur une source hors-sujet** (ui/server.py) — corrigé : masqué quand `pertinence_non_garantie == True`.
3. **L'UI affichait un statut « vert » en bout de chaîne « pertinence non garantie**
4. **0 réponse »** (ui/index.html) — corrigé : message explicite « ⚠ Le système n'a pas pu répondre depuis une source fiable... », statut `NO_ANSWER` rendu en orange (distinct du vert `VALIDATED` et du rouge `BLOCKED`).

Suite complète : 141 tests passants après correction.

Structurellement non résoluble par du code seul

- **D1-D3, C3, E3** — la spec elle-même déclare ces pièges limite mathématique/humaine du contrôle déterministe (§10) : interprétation déguisée, synthèse fallacieuse, fausse attribution, mauvaise juridiction, référent manquant nécessitent soit un jugement humain par

construction, soit un vrai tour de dialogue, soit un corpus multi-juridictionnel réel.

Limites de couverture des sources (pas des bugs)

- **Jurisprudence** (Cour de cassation, arrêts + moyens) absente du corpus — Moulineuse expose Légifrance (codes/JORF) + parlementaire, pas les arrêts. Brancher Judilibre / Légifrance jurisprudence serait une extension.
- **XLSX/XLS et ressources pointant vers des pages HTML** non couverts par `FileRetrievalProvider` (CSV/ZIP-CSV uniquement).

La calibration inter-annotateur (§12) a désormais son outillage complet (`calibration.py`, kappa de Cohen testé) ; il ne reste que l'activité humaine elle-même (annoter un sous-ensemble du gold standard avec 2+ annotateurs réels), qui n'est plus un manque de code.

Gemini, Mistral et LiteLLM ont tous été confirmés fonctionnels de bout en bout (décomposition → génération de claims → AUTHENTIFIÉ) en conditions réelles.

Historique des corrections notables

- **Bug de troncature (B2)** : `_detect_adjacent_truncation` ne vérifiait que la première occurrence d'une citation répétée dans un passage, pouvant manquer une troncature sur la bonne occurrence. Corrigé pour vérifier toutes les occurrences.
- **APIs Gemini/Mistral cassées** : URLs et modèles inventés, jamais vérifiés en direct (contrairement à Moulineuse/data.gouv). Corrigées contre les vraies API (endpoints, modèles actuels, format de payload), plus un fix générique de parsing JSON enveloppé en markdown et la désactivation du mode "thinking" de Gemini 2.5 qui tronquait les réponses courtes.
- **Filtre multi-saut trop large** : `select_next_hop` suivait initialement des renvois MODIFIE vers des ordonnances JORF non récupérables par la route `code_article`. Corrigé en filtrant aussi sur `@naturetexte == "CODE"`.
- **Conflit SentinelGuard.confidential** : un paramètre configurable produisait des entrées de log rejetées par le `SovereignLogStore` (qui refuse toute entrée avec query par construction). Résolu en supprimant l'option — utiliser `SentinelGuard` implique le mode souverain par construction.
- **Vestige tool_choice incohérent (§6)** : `PromptBasedDecomposer/`
`PromptBasedIntentGenerator` calculaient `tool_choice="required"` if `supports_forced_tool_calling` else `None` tout en passant systématiquement `tools=[]` — forcer un outil qui n'est jamais déclaré n'a aucun effet. Nettoyé : ces classes n'appellent jamais MCP elles-

mêmes (l'orchestrateur le fait toujours en amont), donc `tool_choice=None` sans condition, avec le choix architectural documenté en commentaire.

- **LiteLLM bloqué par l'environnement puis débloqué (§17) :**

`litellm.completion()` échouait en SSL: `CERTIFICATE_VERIFY_FAILED` via `httpx/certifi`, alors que les wrappers `urllib` de `gemini.py/mistral.py` fonctionnaient. Cause confirmée : Avast Antivirus fait de l'inspection HTTPS et installe son certificat racine (CN=Avast Web/Mail Shield Root) dans le magasin Windows natif (Cert:\LocalMachine\Root, utilisé par `urllib`), jamais dans le bundle Mozilla embarqué de `certifi` (utilisé par `httpx`). `SSL_CERT_FILE` ne suffisait pas. Débloqué avec le paquet `truststore`, qui patche `ssl` pour utiliser l'API native de l'OS (SChannel sous Windows) au lieu du bundle `certifi` — portable sur toute machine avec inspection HTTPS (antivirus ou proxy d'entreprise), sans export manuel de certificat. `LiteLLMModelProvider` l'injecte automatiquement à l'initialisation. Vérifié en direct sur Mistral via LiteLLM.

Delta v4 — mode document (§7ter) implémenté

Le journal de modifications v3 → v4 de la spec est couvert point par point (16 tests dédiés, `tests/test_document.py`, exemple exécutable `examples/run_document_mode.py`) :

- **§7ter / §15** — `DocumentMode` (analyse | synthèse | production), `DocumentDraft`, `CoverageMap` dans `types.py` ; `document.py` porte `verify_document`, `segment_source_units`, `check_documentary_coverage`. Chaque claim est vérifié individuellement par le vérificateur §7 existant (ancres dures et re-contrôle d'abrogation inclus).
- **INV-015** — `DocumentVerificationResult` n'expose AUCUN statut agrégé : la mosaïque des statuts vit claim par claim ; `publishable` est une porte de conformité (refus §7bis ou violation INV-017), pas un label de qualité, et n'exempte jamais de la validation humaine en risque élevé.
- **INV-016 / §2** — plancher élevé inconditionnel en mode production ; élevé en synthèse d'une source normative (`source_type == "normatif"`) ; cumulé aux conditions communes (statuts faibles, troncature), jamais en remplacement. Contre-exemple testé : synthèse d'une donnée tracée exacte → risque faible.
- **INV-017 / B5** — la segmentation en unités structurales est faite par le CODE (en-têtes « Article/Chapitre/Section/Titre/Livre/Annexe », repli paragraphes), jamais par le LLM — sinon le périmètre de couverture serait manipulable. Le mapping fourni doit : porter exactement les unités du code, couvrir ou déclarer omise chaque unité, ne référencer que des claims existants (lookup). Une synthèse sans mapping n'est

pas publiable ; le cas canonique de la spec (n sections, n-1 mappées, zéro omission déclarée → blocage) est testé.

- **§12 (v4)** — `run_document_measurement` + `DocumentMeasurementReport` : taux de sur-refus et de blocage correct calculés PAR MODE (`over_refusal_rate_by_mode`), démontré par un banc où la synthèse a du sur-refus sans que la production en ait. Le jeu de référence `DOCUMENT_TRAP_DATASET` (`trap_dataset.py`) contient les documents pièges par mode exigés par la spec — rapport avec chapitre omis silencieusement (B5), synthèse sans mapping (INV-017), amendement citant un texte inexistant, plus les cas répondables correspondants : 100% de blocage correct, 0% de sur-refus, sur chacun des trois modes (`python examples/run_measurement.py`).
- **§8** — `governance_version` passe à "v4".

Limite assumée (documentée dans `document.py`) : le lookup d'existence de l'article/alinea visé par un amendement (mode production) exige une récupération — il relève de l'appelant via les routes §6bis ; le plancher élevé inconditionnel garantit qu'aucune production ne contourne l'humain en attendant.

Tests en direct du démonstrateur (2026-07-02) — boucle humaine validée

Deux scénarios joués en conditions réelles (Moulineuse + Mistral) après la Relecture 2, validant que le contrôle de l'information revient effectivement à l'humain :

1. **QOSD n° 812 avec prémisse fausse (A2)** — la question affirmait que la QOSD 812 portait sur la fermeture d'une trésorerie ; le vrai texte porte sur les contrats aidés (Mme Obono). Résultat : texte officiel récupéré tel quel, aucune commune inventée (`NO_ANSWER`), risque élevé (source non opposable + 3 intentions sur 1 requête + couverture 79%), panneau de validation humaine affiché sur chaque intention. Aucune hallucination.
2. **Article 1103 + bonne foi (E1)** — 2 intentions, couverture 100%, requête unique servant les deux. Intention 1 : verbatim exact AUTHENTIFIÉ sur source opposable en vigueur. Intention 2 : passage authentique mais hors sujet (la bonne foi est à l'art. 1104) — **bloqué** par le double plancher query-partagée (E1) + slot inféré (A3), délégué à l'humain. C'est précisément le cas visé par la Relecture 2 : « bonne réponse à côté de la question », publiable automatiquement avant, décision humaine désormais.

Corrections UI issues de ces tests (`ui/server.py`) : - **Claim de contrôle tronqué sur abréviation** : l'extraction de première phrase coupait sur « M. le ministre » → « Mme Danièle Obono interroge M ». Corrigé : les points d'abréviation (M.,

Mme, art., n°, ...) ne terminent plus une phrase. - **Détection du nom de code trop gourmande** : « code civil et quelle est la règle... » capturait toute la fin de la phrase comme titre de code → `RetrievalError`. Corrigé : coupure au premier marqueur de nouvelle proposition, sans casser les titres réels contenant « et » (« code de la construction et de l'habitation »). - **Préposition avalée** : « code de la construction... » était prérempli « code la construction... », cassant le LIKE sur le titre officiel. Corrigé. - **Articles à préfixe tronqués** : « article L. 1232-6 » était détecté « L. ». Corrigé (préfixe L/R/D + point + espace optionnels).

Relecture 2 — 7 angles morts trouvés, 6 corrigés + 1 borné

Seconde relecture systématique (spec entière + tous les modules), corrections avec tests de non-régression (16 nouveaux tests, suite à 157) :

1. **Plancher §2 incomplet (INV-011) — orchestration.py** : le plancher ne recevait que 4 conditions ; le statut des claims n'y figurait pas. Une paraphrase INTERPRÉTATION (ou un verbatim CITÉ_NON_OPPOSABLE) pouvait être publiée en risque faible sans validation humaine, alors que la matrice §10 classe B3/D1/F1 « borné puis délégué à l'humain ». Corrigé : tout claim INTERPRÉTATION ou CITÉ_NON_OPPOSABLE déclenche le plancher → risque élevé → validation humaine (§4 étape 9).
2. **Ancrage lexical aveugle à la négation et aux chiffres (verifier.py)** : ne/pas étant des stopwords, « le contrat n'est pas valide » s'ancrait comme « le contrat est valide » ; un chiffre substitué (« 14 jours » pour « 10 jours ») se noyait dans le seuil de 60%. Corrigé par des **ancres dures** : tous les marqueurs de négation et tous les tokens chiffrés de la reformulation doivent exister dans la source (« n' » élidé ≡ « ne »). Limite documentée : la direction inverse (le passage nie, la reformulation affirme) reste indétectable lexicalement — couverte par le point 1 (toute INTERPRÉTATION passe désormais par un humain).
3. **opposable_override piloté par la requête (moulineuse.py)** : la route pastille laissait la query rendre un amendement opposable — INV-010 contournable par un flag le jour où la formulation de requête (§4 étape 3) est déléguée à un LLM. Corrigé : override supprimé, un article pastillé n'est jamais opposable (l'opposabilité dérive du type de document, point final).
4. **« 1 intention → 1 requête » structurellement creux (orchestration.py)** : `ask(message, query)` réutilise UNE query pour toutes les intentions issues de la décomposition — N questions → 1 requête, le piège E1 recréé en interne. Borné (pas résolu) : tant que l'étape 3 (formulation de requête par intention) n'est pas implémentée, N>1 intentions avec query partagée force le risque élevé pour toutes →

validation humaine. La vraie correction reste l'implémentation de l'étape 3.

5. **Détection d'abrogation sans défense en profondeur (verifier.py)** : seul `code_article` consultait le cycle de vie ; un Passage mal construit par un provider (`opposable=True`, `etat=ABROGE`) passait AUTHENTIFIÉ. Corrigé : le vérificateur re-contrôle `metadata["etat"]` — tout état $\neq$ VIGUEUR force CITÉ_NON_OPPOSABLE au mieux, quel que soit le provider.
6. **Sélection silencieuse en cas d'ambiguïté (moulineuse.py + orchestration.py)** : `rows[0]` sur un LIKE `%civil%` matchant deux codes distincts pouvait servir le mauvais article, verbatim exact, AUTHENTIFIÉ (`candidate_count` était stocké mais jamais lu). Corrigé : `selection_ambiguous` posé quand plusieurs **textes distincts** matchent (plusieurs versions du même texte ne sont pas ambiguës), lu par l'orchestration → risque élevé. Idem route `texte_libre` (hits multiples).
7. **Sur-refus de normalisation (§7/INV-013)** : comparaison verbatim sensible à la casse et à la ponctuation de bord (« Les contrats » cité « les contrats » → NON_AUTHENTIFIÉ), et `normalize_numeric` refusait « 14,5 » face à « 14.5 ». Corrigé : comparaison casefold + rognage de la ponctuation de bord de la citation (la détection de troncature suit les mêmes règles), virgule décimale normalisée en point. L'anti-épissage reste intact : le contrôle d'ellipse porte sur le claim brut. Unités et arrondis (INV-013) restent non traités.

Relecture complète — 6 angles morts trouvés et corrigés

Relecture systématique de tous les modules, chaque bug confirmé par un test reproductible en direct puis corrigé avec test de non-régression :

1. **visited_documents partagé entre intentions (orchestration.py)** — deux questions légitimes sur le même article faisaient planter tout le pipeline en Document `already visited`. Confusion entre INV-006 (borner le multi-saut *dans* une intention) et INV-002 (étanchéité *entre* intentions). Corrigé : l'état de récupération est réinitialisé par intention, le budget global reste partagé.
2. **Couverture aveugle aux chiffres courts (coverage.py)** — `len(t) > 1` filtrait tous les tokens d'un caractère, rendant "article 6" invisible au contrôle E4 (`ratio=1.0` malgré une intention oubliée). Corrigé : les chiffres isolés sont désormais significatifs.
3. **A3 désarmé sur numéros courts (slot_provenance.py)** — test de sous-chaîne brute : l'article "16" inféré était déclaré "copié" car "16" $\subset$ "2016". Corrigé : comparaison par mot entier (tokenisation), en préservant la tolérance au formatage variable ("L. 1232-6" $\leftrightarrow$ "L1232-6") et les slots multi-mots.

4. **Faux positifs de troncature (verifier.py)** — `startswith(connector)` sans frontière de mot : "sauferie" déclenchait le drapeau "sauf".
Corrigé : vérification de frontière (espace/punctuation/fin de chaîne) après le connecteur.
5. **Refus jamais journalisé (audit.py + orchestration.py)** —
`verify_claims` levait une exception, donc un refus (§7bis) plantait `Orchestrator.run` avant journalisation et `compliance_status="BLOCKED"` était du code mort. Corrigé : l'exception `VerificationError` porte désormais le `VerificationResult` complet, et l'orchestrateur capture le refus pour en faire un résultat `BLOCKED` journalisable à risque élevé — les autres intentions continuent (schéma §4 étape 8).
6. **McpToolClient non robuste à l'inspection SSL (mcp_client.py)** — le SDK `mcp` utilise `httpx`, donc les appels `Moulineuse/data.gouv` échouaient en `CERTIFICATE_VERIFY_FAILED` dès qu'Avast inspectait le HTTPS. Corrigé : `ensure_system_trust_store()` factorisé dans `trust.py` et injecté par `McpToolClient` comme par `LiteLLMModelProvider`.
`Moulineuse` révérifié en direct.

Extensions issues des tests utilisateur

- **Route question parlementaire (§6bis/§6ter) :**
`MoulineuseRetrievalProvider.route="parlement_question"` via `get_parlement_item` (QE/QOSD/QG). Non opposable par nature → `CITÉ_NON_OPPOSABLE`, jamais AUTHENTIFIÉ. Testé en direct sur la QOSD n°812 : le système a démontré le piège A2 (prémisse fausse) en refusant d'inventer une commune absente du texte officiel.
- **Faux négatif sur les paraphrases fidèles corrigé (§5/§7) :** le vérificateur exigeait un verbatim exact et bloquait toute reformulation correcte en `NON_AUTHENTIFIÉ`. Corrigé sur deux volets : (1) le prompt B distingue désormais citation exacte (AUTHENTIFIÉ) et reformulation (INTERPRÉTATION) ; (2) le vérificateur accepte une INTERPRÉTATION si ses termes sont **ancrés** dans le passage (recouvrement lexical déterministe ≥60%), sinon c'est une invention pure → `NON_AUTHENTIFIÉ`. Le mot AUTHENTIFIÉ reste réservé au verbatim exact (§7). Découvert parce qu'un vrai LLM produit des paraphrases, ce que les mocks ne montraient pas.
- **FileRetrievalProvider pour les ressources non-tabulaires (§6ter) :** la plupart des ressources INSEE ne sont PAS dans l'API tabulaire de `data.gouv` (elles pointent vers des fichiers/pages externes). Ce provider télécharge le fichier, détecte défensivement son vrai format (magic bytes — le MIME déclaré ment : INSEE annonce `text/csv` pour un ZIP), extrait le CSV de données (refuse si ambigu), et adresse une cellule par filtres multi-colonnes (refus si 0 ou >1 ligne). Testé en direct : 891 naissances (dép. 06, juillet 2025) → `DONNÉE_TRACÉE`. XLSX/HTML non couverts (limite

documentée). Branché dans MultiSourceRetrievalProvider (clé `filters`).

- **Démonstrateur web (ui/)** : serveur `http.server` `stdlib` (zéro dépendance web) + page HTML unique. Champ question → décomposition Mistral → récupération réelle (4 routes normatives + 2 routes donnée) → vérification → statuts colorés + journal §8.
Lancement : `python ui/server.py` puis `http://localhost:8765`. **Marquage « intervention humaine requise » (§4 étape 9)** : tout résultat à risque élevé porte un badge explicite avec les motifs possibles et sa clé de validation (`intent_id`, `passage_hash`) ; il reste « NON PUBLIABLE en l'état ». L'UI ne porte pas la décision (le panneau Approuver/Rejeter et `POST /validate` d'une première itération ont été retirés) : l'approbation/rejet relève du circuit de validation de l'institution, via le `HumanValidationRegistry` du cœur — mécanique inchangée et testée (`test_ask_publishes_after_human_approval`). Quand le LLM ne produit rien (sur-refus fréquent de `mistral-small`, §12), un claim de contrôle déterministe extrait du passage démontre visuellement la garantie du vérificateur.